INDUS11

AI Financial Risk & Fraud Decision Engine

TECHNICAL PROGRESS REVIEW · 25 July 2026

Joshi Om · Krish Gajera · Drashti Dedaniya

Semester Group Project · CHARUSAT — DEPSTAR

01 · FEEDBACK

Mentor feedback — and what we changed

Feedback received

- Show real technical progress, not just the project idea
- Present a concrete system architecture and module design
- Add a process-flow / data-flow diagram
- Compare existing solutions in a literature review
- Provide implementation evidence — screenshots, code, structure
- Set up a GitHub repo with team + mentor as collaborators

Improvements made

- Moved from concept to a running end-to-end prototype
- Documented the 5-layer architecture and how modules interact
- Added a step-by-step data-flow diagram of the pipeline
- Built a comparison of commercial systems, backed by research
- Live dashboard + 12 REST APIs running on real data (shown here)
- Repository organised for GitHub with structured commits

What we are solving

The problem

Banks must approve or block a payment in milliseconds.

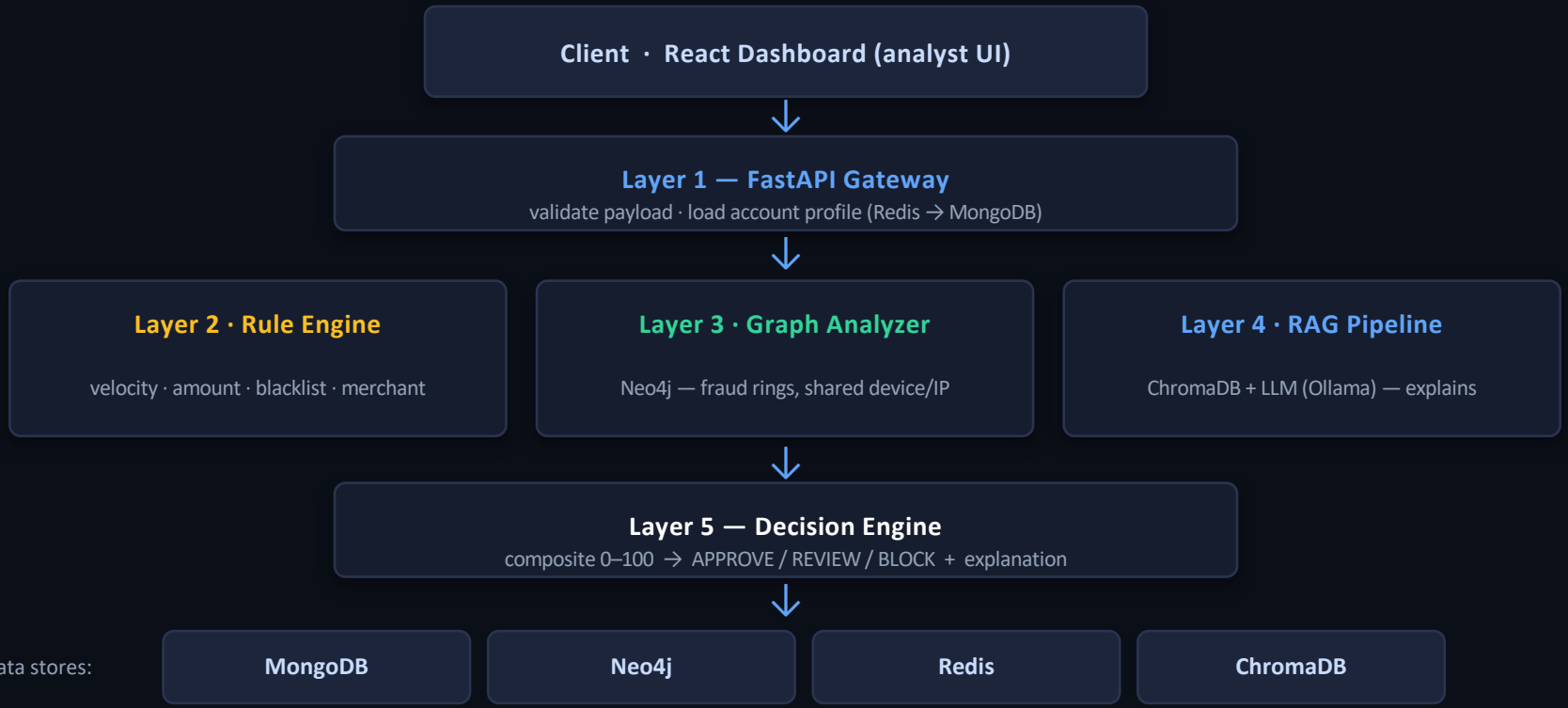
Rule-only systems miss new and coordinated fraud; pure machine-learning systems cannot explain why they blocked a customer.

Analysts and regulators need a clear reason behind every decision.

Our objectives

- Score each transaction 0–100 and return APPROVE / REVIEW / BLOCK
- Combine three engines — rules, graph analysis, and a RAG-LLM
- Attach a plain-English explanation to every decision
- Detect fraud rings that single-transaction checks cannot see
- Deliver a live dashboard for analysts, running on one machine

System architecture — 5-layer pipeline



04 · PROCESS FLOW

Data-flow — from transaction to decision

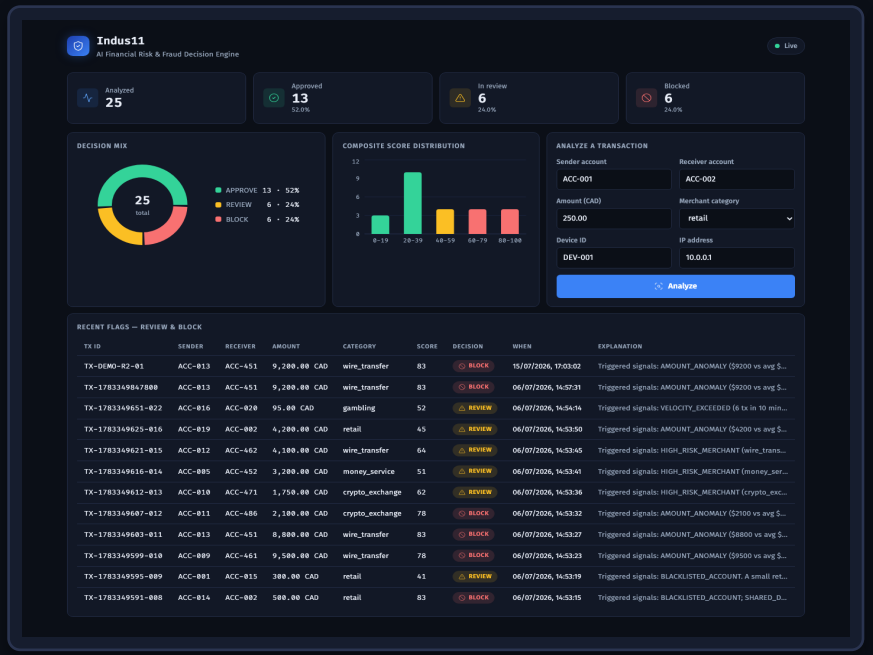
1. **Client submits a transaction** POST /api/v1/transactions/analyze (sender, receiver, amount, device, IP)
2. **Validate & load context** Pydantic schema check · fetch account profiles from Redis, fallback MongoDB
3. **Run three engines concurrently** Rule Engine + Neo4j Graph Analyzer + RAG Pipeline via asyncio.gather()
4. **Aggregate the scores** Decision Engine sums rule (0–40) + graph (0–30) + AI (0–30) = 0–100
5. **Decide & explain** map to APPROVE (0–39) / REVIEW (40–69) / BLOCK (70+) + plain-English reason
6. **Persist & respond** save the record to MongoDB · return JSON (decision, score, breakdown)
7. **Dashboard reads results** stats endpoints feed live charts and the flagged-transactions table

Existing solutions — and where we differ

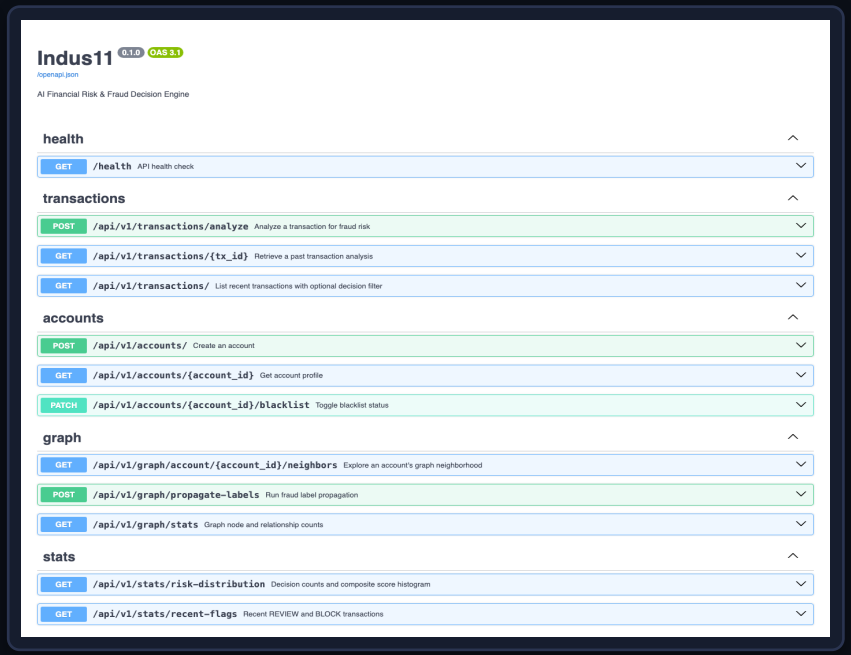
System	Approach	Explains why?	Open / on-prem
FICO Falcon	Neural networks on shared bank data	Limited — black box	No — proprietary
Feedzai	ML risk platform (RiskOps)	Limited	No — commercial
Featurespace ARIC	Adaptive behavioural analytics	Limited	No — commercial
Traditional rule engines	Static hand-written rules	Yes, but rigid	Varies
Indus11 (ours)	Rules + graph + RAG-LLM ensemble	Yes — every decision	Yes — open, local

Research basis: the APATE study (Van Vlasselaer et al., 2015) shows combining graph features with traditional features reaches AUC > 0.98 — evidence that our rules-plus-graph design outperforms either alone.

Live system — dashboard & working API



React dashboard — live risk metrics, decision mix, flagged transactions



FastAPI — 12 documented REST endpoints (health, transactions, accounts, graph, stats)

Codebase, code & technology

Project structure

```
app/
  main.py          FastAPI app + startup
  config.py        settings (.env)
  api/routes/      transactions · accounts
                   graph · stats · health
  core/            db · neo4j · redis · rate
  models/          Beanie documents
  schemas/         Pydantic contracts
  services/        rule_engine · graph_
                   analyzer · rag_pipeline
                   · decision_engine
dashboard/         React + Vite + Recharts
scripts/           seed_mongo · seed_neo4j
tests/             test_fraud.py (8 tests)
docker-compose.yml
```

Decision Engine — composite scoring

```
composite = rule.score + graph.score + rag.score
composite = min(composite, 100)

if   composite >= BLOCK_THRESHOLD:   # 70+
     decision = Decision.BLOCK
elif composite >= REVIEW_THRESHOLD:  # 40–69
     decision = Decision.REVIEW
else:
     decision = Decision.APPROVE     # 0–39
```

Live example $9,200 wire → known fraud ring

BLOCK **83/100** = rule 25 + graph 30 + AI 28

Stack: Python · FastAPI · MongoDB · Neo4j · Redis · ChromaDB · LangChain + Ollama · React · Docker · pytest (8/8 passing)

What was hard, and what is next

Challenges faced

- Coordinating four data stores (Mongo, Neo4j, Redis, Chroma) locally
- Keeping the LLM output consistent — solved with temperature 0 and a 30-point cap
- Writing Cypher queries to detect circular money-mule flows
- Migrating the data layer from MySQL to MongoDB for schema flexibility

Future work

- Measure accuracy on the synthetic dataset (precision / recall)
- Train a supervised model as a fourth scoring signal
- Add fraud-ring visualisation to the dashboard
- Harden the API — authentication, encryption, load testing
- Complete the SRS and the final project report

Where the project stands

A working end-to-end prototype

All five layers run together and score real transactions.

Live dashboard + 12 REST APIs

Analysts see decisions, charts, and flagged transactions in real time.

Graph + AI, explained

Fraud rings caught by Neo4j; every decision carries a plain-English reason.

On track for the next milestones

Next: accuracy metrics, hardening, SRS, and the final report.